word2vec-rs

Implementation Deep Dive

A complete walkthrough of a Word2Vec library
written from scratch in Rust — every algorithm, every function,
every design decision explained.

Author	Chaganti Reddy
Language	Rust 2021 Edition
Crate	`word2vec v0.1.0`
License	MIT

This document assumes you understand what word embeddings are
at a high level. Everything else is explained from first principles.

Table of Contents

1 What is Word2Vec? ... 4
- 1.1 Why vectors at all? ... 4
- 1.2 The two architectures ... 4
- 1.3 Training objective ... 4

2 Project Structure ... 6

3 Module: `error.rs` ... 7
- 3.1 Purpose ... 7
- 3.2 The `thiserror` crate ... 7
- 3.3 The `Result` type alias ... 7

4 Module: `config.rs` ... 8
- 4.1 Every field explained ... 8
- 4.2 `Config::validate()` ... 9

5 Module: `vocab.rs` ... 10
- 5.1 The `Vocabulary` struct ... 10
- 5.2 `Vocabulary::build()` ... 10
- 5.3 Subsampling: `should_subsample()` ... 11
- 5.4 The Unigram Noise Table ... 11
- 5.5 `tokenise_and_subsample()` ... 12

6 Module: `model.rs` ... 13
- 6.1 Weight matrices ... 13
- 6.2 Initialisation: `Model::new()` ... 13
- 6.3 The sigmoid function ... 14
- 6.4 Skip-gram update: `skipgram_update()` ... 14
- 6.5 CBOW update: `cbow_update()` ... 15
- 6.6 `sentence_to_pairs()` ... 15

7 Module: `trainer.rs` ... 17
- 7.1 `Trainer::train()` ... 17
- 7.2 Linear learning rate decay ... 17
- 7.3 Sentence shuffling ... 17
- 7.4 `EpochStats` ... 18
- 7.5 Progress bars ... 18

8 Module: `embeddings.rs` ... 19
- 8.1 `get_vector()` ... 19
- 8.2 `similarity()` ... 19
- 8.3 `most_similar()` ... 19
- 8.4 `analogy()` ... 20

8.5 Save / Load 20
9 Module: `plot.rs` 21
9.1 `plot_loss_curve()` 21
9.2 `plot_word_vectors_pca()` 21
10 Binaries 22
10.1 `bin/train.rs` — CLI Trainer 22
10.2 `bin/evaluate.rs` — Interactive REPL 22
11 Testing Strategy 23
11.1 Unit tests (35 total) 23
11.2 Integration tests (10 total) 23
11.3 Doc tests (14 total) 23
12 CI / CD Pipeline 24
12.1 Job 1: Check (every push and PR) 24
12.2 Job 2: Docs + Artifacts (main branch only) 24
12.3 Job 3: Deploy (main branch only, after Job 2) 24
13 Key Rust Concepts Used 25
13.1 Ownership and borrowing 25
13.2 `Vec<f32>` for matrices 25
13.3 `Option<T>` and `Result<T, E>` 25
13.4 Traits 25
13.5 `SmallRng` 25
14 Training Results 26
15 What to Do Next 27
15.1 Train on a real corpus 27
15.2 Evaluate on standard benchmarks 27
15.3 Add parallelism 27
15.4 Use embeddings in Python 27
16 Summary 28

1 What is Word2Vec?

Word2Vec is a neural network technique introduced by Tomas Mikolov et al. at Google in 2013. Its goal is deceptively simple: given a large text corpus, produce a dense numerical vector for every word in the vocabulary such that **semantically similar words have vectors that are close together**.

The key insight is the **distributional hypothesis** — words that appear in similar contexts tend to have similar meanings. "Cat" and "dog" both appear near words like "pet", "feed", "fur", "bark", "meow". So their vectors should end up similar.

1.1 Why vectors at all?

Computers cannot work with words directly. The naive approach is **one-hot encoding**: a vector of length V (vocabulary size) with a single 1 at the word's index and zeros everywhere else. This is terrible for two reasons:

- It is **sparse** — for a vocabulary of 50,000 words, each vector has 49,999 zeros.
- It captures **no meaning** — every pair of words is equally distant. "Cat" and "dog" are no more similar than "cat" and "democracy".

Word2Vec produces **dense** vectors (typically 50–300 dimensions) where the geometry encodes meaning. The famous example:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

This arithmetic actually works on well-trained vectors because the offset from "man" to "woman" (a gender direction) is the same as the offset from "king" to "queen".

1.2 The two architectures

Word2Vec comes in two flavours, both implemented in this codebase:

Skip-gram

Given a center word, predict its surrounding context words.

Better for rare words and larger datasets. This is the more commonly used variant.

CBOW (Continuous Bag of Words)

Given surrounding context words, predict the center word.

Trains faster, works better on smaller datasets and frequent words.

1.3 Training objective

Both architectures use **Negative Sampling** to make training tractable. Instead of doing a full softmax over the entire vocabulary (which would require computing a dot product with every word at every step), we:

1. Treat the (center, context) pair as a **positive** example.
2. Sample k random **negative** words from the vocabulary.

3. Train a binary classifier: positive pair scores high, negative pairs score low.

The loss for one positive pair with k negatives is:

$$\mathcal{L} = \log \sigma(\boldsymbol{v}_o \cdot \boldsymbol{v}_c) + \sum_{i=1}^{k} \log \sigma\left(-\boldsymbol{v}_{n_i} \cdot \boldsymbol{v}_c\right)$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function, $\boldsymbol{v}_c$ is the center word vector, $\boldsymbol{v}_o$ is the context word vector, and $\boldsymbol{v}_{n_i}$ are the negative sample vectors.

2 Project Structure

The codebase is a standard Rust library (`--lib`) with two binary entrypoints and a separate integration test file:

```
word2vec/
├── Cargo.toml              — dependencies and binary declarations
├── src/
│   ├── lib.rs              — crate root, public API re-exports
│   ├── error.rs            — unified error type
│   ├── config.rs           — hyperparameter struct
│   ├── vocab.rs            — vocabulary construction
│   ├── model.rs            — neural network weights and SGD
│   ├── trainer.rs          — training loop
│   ├── embeddings.rs       — post-training inference API
│   ├── plot.rs             — PNG chart generation
│   └── bin/
│       ├── train.rs        — CLI training binary
│       └── evaluate.rs     — interactive query REPL
├── tests/
│   └── integration.rs      — end-to-end tests
└── examples/
    └── basic_training.rs   — runnable full example
```

Each `src/*.rs` file is a **module** — Rust's unit of code organisation. They are declared in `lib.rs` with `pub mod` and re-exported for external users.

3 Module: `error.rs`

```rust
pub enum Word2VecError {
    EmptyVocabulary,
    UnknownWord(String),
    InvalidDimension(usize),
    // ...
}
```

3.1 Purpose

In Rust, errors are values — they are returned, not thrown. Every function that can fail returns `Result<T, E>` where `E` is the error type. Having a single unified error enum means callers only need to handle one error type no matter which part of the library they use.

3.2 The `thiserror` crate

Writing error types by hand requires implementing several standard traits (`Display`, `Error`, `From`). The `thiserror` crate generates this boilerplate from a simple `#[derive(Error)]` annotation and `#[error("message")]` attributes on each variant. For example:

```rust
#[derive(Debug, Error)]
pub enum Word2VecError {
    #[error("Word not found in vocabulary: `{0}`")]
    UnknownWord(String),

    #[error("I/O error: {0}")]
    Io(#[from] std::io::Error),
}
```

The `#[from]` annotation on `Io` means that any `std::io::Error` can be automatically converted into a `Word2VecError::Io` — so you can use the `?` operator on file operations without manual wrapping.

3.3 The `Result` type alias

```rust
pub type Result<T> = std::result::Result<T, Word2VecError>;
```

This type alias means every module can write `Result<Vocabulary>` instead of the verbose `std::result::Result<Vocabulary, Word2VecError>`. It is a common Rust pattern.

4 Module: `config.rs`

```rust
pub struct Config {
    pub embedding_dim: usize,
    pub window_size: usize,
    pub negative_samples: usize,
    pub epochs: usize,
    pub learning_rate: f32,
    pub min_learning_rate: f32,
    pub min_count: usize,
    pub subsample_threshold: f64,
    pub model: ModelType,
    pub num_threads: usize,
    pub seed: u64,
}
```

4.1 Every field explained

`embedding_dim` — The number of dimensions in each word vector. Typical values are 50, 100, 200, or 300. Larger dimensions capture more nuance but require more memory and training time. With a small corpus, 50–100 is sufficient.

`window_size` — How many words to look at on each side of the center word. A window of 5 means we consider up to 5 words to the left and 5 words to the right. Wider windows capture more topical/document-level relationships. Narrow windows (2–3) capture tighter syntactic relationships.

`negative_samples` — How many random "wrong" words to contrast against each positive (center, context) pair. Higher values give a better gradient signal but slow down training. The original paper recommends 5–20 for small datasets, 2–5 for large ones.

`epochs` — How many full passes over the training corpus. One epoch means the model sees every sentence once. More epochs generally improve quality up to a point.

`learning_rate` / **`min_learning_rate`** — The learning rate controls how large each gradient update step is. We start at `learning_rate` (typically 0.025) and linearly decay it to `min_learning_rate` (0.0001) over the full training run. This is called **linear decay scheduling** — early in training the model makes big moves to find a good region of parameter space, then fine-tunes with smaller steps.

`min_count` — Words appearing fewer than this many times are discarded from the vocabulary. Rare words (typos, proper nouns, junk) add noise and waste computation. A `min_count` of 5 is a common default for large corpora.

`subsample_threshold` — Controls stochastic downsampling of frequent words (see `vocab.rs` for the formula). Very common words like "the", "a", "is" appear so often that training on every occurrence provides diminishing returns. We randomly skip them with probability proportional to their frequency. The original paper suggests values between `1e-3` and `1e-5`.

`model` — An enum: `ModelType::SkipGram` or `ModelType::Cbow`.

`seed` — The random seed for reproducibility. Setting this ensures that two training runs with identical config and corpus produce identical results.

4.2 `Config::validate()`

```
pub fn validate(&self) -> Result<(), String> {
    if self.embedding_dim == 0 {
        return Err(format!("embedding_dim must be > 0"));
    }
    // ...
}
```

Called at the start of training to catch configuration mistakes early, before any computation begins.

5 Module: `vocab.rs`

The vocabulary module is one of the most important — it transforms raw text into the integer indices that the neural network actually operates on.

5.1 The `Vocabulary` struct

```
pub struct Vocabulary {
    pub word2idx: HashMap<String, usize>,
    pub idx2word: Vec<String>,
    pub counts:   Vec<u64>,
    pub noise_table: Vec<u32>,
    pub total_tokens: u64,
}
```

- `word2idx` — a hash map from word string to its integer index. O(1) lookup.
- `idx2word` — the reverse: given an index, get the word back. Just a `Vec`.
- `counts` — the raw frequency of each word, indexed in the same order as `idx2word`.
- `noise_table` — a precomputed table for fast negative sampling (explained below).
- `total_tokens` — total number of tokens in the corpus (after filtering), used to compute relative frequencies.

5.2 `Vocabulary::build()`

This is the constructor. It takes the tokenised corpus (a `Vec<Vec<String>>` — a list of sentences, each sentence a list of words) and a `Config`.

Step 1 — Count frequencies:

```
let mut raw_counts: HashMap<String, u64> = HashMap::new();
for sentence in sentences {
    for token in sentence {
        *raw_counts.entry(token.clone()).or_insert(0) += 1;
    }
}
```

Iterates every token in every sentence, building a frequency map.

Step 2 — Apply `min_count` filter:

```
let filtered: Vec<(String, u64)> = raw_counts
    .into_iter()
    .filter(|(_, c)| *c >= config.min_count as u64)
    .collect();
```

Drops any word that appears fewer than `min_count` times.

Step 3 — Sort by descending frequency:

```
filtered.sort_unstable_by(|a, b|
    b.1.cmp(&a.1).then(a.0.cmp(&b.0))
);
```

Most frequent words get lower indices. This is important because the noise table construction (below) assumes this ordering. Ties are broken alphabetically for determinism.

Step 4 — Build the noise table.

5.3 Subsampling: `should_subsample()`

Frequent words like "the" appear so often that the model wastes time training on them. Mikolov's subsampling formula:

$$P(\text{keep}) = \min 1, \sqrt{\frac{t}{f}} + \frac{t}{f}$$

Where t is `subsample_threshold` (e.g. `1e-3`) and f is the word's relative frequency in the corpus. Words with $f \gg t$ get discarded with high probability. Words with $f \leq t$ are always kept.

```
pub fn should_subsample(&self, idx: usize, threshold: f64, dice: f64) -> bool {
    let freq = self.counts[idx] as f64 / self.total_tokens as f64;
    let keep_prob = ((threshold / freq).sqrt() + threshold / freq).min(1.0);
    dice >= keep_prob   // true = discard
}
```

`dice` is a uniform random number in [0, 1). If `dice >= keep_prob`, the word is discarded from the training sentence.

5.4 The Unigram Noise Table

Negative sampling requires drawing random words from the vocabulary — but not uniformly. If we sampled uniformly, common words like "the" would appear as negatives constantly, and the model would learn to push them away regardless of context, which distorts the embeddings.

Instead, the original paper samples from the **unigram distribution raised to the power 3/4**:

$$P(\text{word}_i) = \frac{f_i^{\frac{3}{4}}}{\sum_j f_j^{\frac{3}{4}}}$$

The exponent 3/4 is a heuristic that downweights very frequent words (making common words less likely to be negatives) while still preferring frequent words over rare ones.

Implementing this efficiently: we precompute a flat array of 1,000,000 word indices, where each index appears proportionally to its `freq^0.75` weight. To draw a negative sample, we just pick a random position in this array — O(1).

```rust
const TABLE_SIZE: usize = 1_000_000;

fn build_noise_table(counts: &[u64]) -> Vec<u32> {
    let powered: Vec<f64> = counts.iter()
        .map(|&c| (c as f64).powf(0.75))
        .collect();
    let total: f64 = powered.iter().sum();

    let mut table = Vec::with_capacity(TABLE_SIZE);
    let mut cumulative = 0.0_f64;
    let mut word_idx = 0usize;

    for i in 0..TABLE_SIZE {
        let threshold = (i as f64 + 1.0) / TABLE_SIZE as f64;
        while cumulative / total < threshold {
            cumulative += powered[word_idx];
            word_idx += 1;
        }
        table.push(word_idx as u32);
    }
    table
}
```

This fills the table using cumulative distribution — it is essentially an inverse CDF lookup precomputed for all 1M positions.

5.5 `tokenise_and_subsample()`

```rust
pub fn tokenise_and_subsample(
    &self, sentence: &[String],
    threshold: f64, rng: &mut SmallRng,
) -> Vec<usize> {
    sentence.iter()
        .filter_map(|w| self.word2idx.get(w).copied())
        .filter(|&idx| !self.should_subsample(idx, threshold, rng.gen()))
        .collect()
}
```

Combines two operations: convert word strings to indices (dropping unknown words), then stochastically discard frequent words. Returns a `Vec<usize>` — the processed token sequence for one sentence ready for the model.

6 Module: `model.rs`

This is the neural network — the weights and the gradient updates.

6.1 Weight matrices

```
pub struct Model {
    pub vocab_size: usize,
    pub embedding_dim: usize,
    pub input_weights:  Vec<f32>,   // W_in:  [vocab_size × embedding_dim]
    pub output_weights: Vec<f32>,   // W_out: [vocab_size × embedding_dim]
}
```

Word2Vec has **two** embedding matrices:

- **`input_weights` (W_in)** — the "input" or center-word embeddings. These are the vectors you use after training — when you call `most_similar("king")`, it looks up row `king_idx` in this matrix.
- **`output_weights` (W_out)** — the "output" or context-word embeddings. These are used during training for the dot-product scoring but are typically discarded afterward. Some applications average W_in and W_out.

Both matrices are stored as flat `Vec<f32>` in row-major order. Row `i` starts at byte offset `i * embedding_dim * 4`. This layout is cache-friendly — accessing all dimensions of one word's vector is a contiguous memory read.

6.2 Initialisation: `Model::new()`

```
pub fn new(vocab_size: usize, embedding_dim: usize, seed: u64) -> Self {
    let r = xavier_range(embedding_dim);  // sqrt(6 / dim)
    let input_weights: Vec<f32> = (0..n)
        .map(|_| rng.gen_range(-r..r))
        .collect();
    let output_weights = vec![0.0_f32; n];
    // ...
}
```

Xavier uniform initialisation sets the initial weights in the range $\left[-\sqrt{\frac{6}{d}}, +\sqrt{\frac{6}{d}}\right]$ where d is the embedding dimension. This keeps the initial variance of dot products in a reasonable range regardless of dimension — if weights were too large, dot products would saturate the sigmoid; if too small, gradients would vanish early in training.

Output weights start at exactly zero. This asymmetry is intentional: the input embeddings start with random signal, while output weights are learned from scratch during training.

6.3 The sigmoid function

```
fn sigmoid(x: f32) -> f32 {
    1.0 / (1.0 + (-x).exp())
}
```

The sigmoid squashes any real number to the range (0, 1), which we interpret as a probability. For large positive x, $\sigma(x) \approx 1$. For large negative x, $\sigma(x) \approx 0$. At $x = 0$, $\sigma(0) = 0.5$.

The gradient of the binary cross-entropy loss with respect to the score x is simply $\sigma(x) - y$, where y is 1 for positive pairs and 0 for negative pairs. This elegant simplification is why sigmoid + binary cross-entropy is so popular.

6.4 Skip-gram update: `skipgram_update()`

Given a center word index, one context word index, and a list of negative sample indices:

```
pub fn skipgram_update(
    &mut self, center: usize, context: usize,
    negatives: &[usize], lr: f32,
) -> f32 {
    let mut grad_input = vec![0.0f32; dim];

    // Positive pair
    let score = dot(input_vec(center), output_vec(context));
    let sig   = sigmoid(score);
    let err   = sig - 1.0;           // gradient w.r.t. score
    for i in 0..dim {
        grad_input[i]          += err * output_vec[i];
        output_weights[i]      -= lr * err * input_vec[i];
    }

    // Each negative pair
    for &neg in negatives {
        let score = dot(input_vec(center), output_vec(neg));
        let sig   = sigmoid(score);  // target is 0, so grad = sig - 0 = sig
        for i in 0..dim {
            grad_input[i]      += sig * output_neg[i];
            output_weights[i]  -= lr * sig * input_vec[i];
        }
    }

    // Apply accumulated gradient to center word
    for i in 0..dim {
        input_weights[i] -= lr * grad_input[i];
    }
}
```

Key points:

- For the positive pair, the error is `sigmoid(score) - 1`. We **want** the score to be high (probability 1), so the gradient pushes the vectors toward each other.
- For negative pairs, the error is `sigmoid(score) - 0 = sigmoid(score)`. We **want** the score to be low (probability 0), so the gradient pushes those vectors apart.

- The gradient with respect to the center word's input vector is **accumulated** across all context and negative updates before being applied. This is correct — the center word participates in all these interactions simultaneously.
- Updates to output vectors are applied **immediately** per pair. This is the standard approach and avoids having to store separate gradients for each output word.

6.5 CBOW update: `cbow_update()`

CBOW is the reverse: given a bag of context word vectors, predict the center word.

```
pub fn cbow_update(
    &mut self, center: usize, context_words: &[usize],
    negatives: &[usize], lr: f32,
) -> f32 {
    // Average the context word vectors
    let scale = 1.0 / context_words.len() as f32;
    let ctx_avg: Vec<f32> = (0..dim)
        .map(|d| context_words.iter()
            .map(|&i| input_weights[i * dim + d])
            .sum::<f32>() * scale)
        .collect();

    // Positive pair: ctx_avg should score high against center
    let score = dot(&ctx_avg, output_vec(center));
    let err   = sigmoid(score) - 1.0;
    // ... update output_weights[center] ...

    // Negative pairs ...

    // Distribute gradient back to ALL context word input vectors
    for &cidx in context_words {
        for i in 0..dim {
            input_weights[cidx * dim + i] -= lr * grad_ctx[i] * scale;
        }
    }
}
```

The key difference from skip-gram: we average the context word vectors into a single `ctx_avg` vector, use that for scoring, then distribute the gradient **back** to all context word embeddings. The `* scale` factor in the backward pass ensures the gradient magnitude is independent of window size.

6.6 `sentence_to_pairs()`

```
pub fn sentence_to_pairs(
    tokens: &[usize], window_size: usize, rng: &mut SmallRng,
) -> Vec<(usize, Vec<usize>)> {
    for (pos, &center) in tokens.iter().enumerate() {
        let win = rng.gen_range(1..=window_size);  // dynamic window
        let start = pos.saturating_sub(win);
        let end = (pos + win + 1).min(tokens.len());
        let context: Vec<usize> = (start..end)
```

```
            .filter(|&i| i != pos)
            .map(|i| tokens[i])
            .collect();
        pairs.push((center, context));
    }
}
```

Dynamic window: rather than always using the full `window_size`, we draw a random window width from `[1, window_size]` for each center word. This means words closer to the center are sampled more often (they appear in more window positions), which is the behaviour described in the original paper. It acts as a soft weighting that emphasises nearby context.

7 Module: `trainer.rs`

The trainer orchestrates everything: vocabulary building, model initialisation, and the training loop.

7.1 `Trainer::train()`

This is the main entry point. The full flow:

```
corpus (Vec<String>)
  → tokenise (split on whitespace)
  → Vocabulary::build (count, filter, sort, noise table)
  → Model::new (Xavier init)
  → for each epoch:
      shuffle sentence order
      for each sentence:
          tokenise_and_subsample (indices + drop frequent words)
          sentence_to_pairs (center/context pairs with dynamic window)
          for each (center, context_words) pair:
              sample k negatives from noise_table
              compute current lr (linear decay)
              Model::update (SGD step)
              accumulate loss
      record EpochStats
  → return Embeddings
```

7.2 Linear learning rate decay

```rust
let progress = global_step as f32 / total_steps.max(1) as f32;
let lr = (lr_start - (lr_start - lr_min) * progress).max(lr_min);
```

`global_step` counts every single (center, context) pair processed across all epochs. At step 0, `lr = lr_start`. At the final step, `lr ≈ lr_min`. The decay is **linear**, not exponential — this matches the original C implementation.

Why decay? Early in training, the weights are random and far from their optimal values, so large steps help explore the parameter space quickly. Late in training, we are near a good solution and want to fine-tune carefully without oscillating.

7.3 Sentence shuffling

```rust
let mut indices: Vec<usize> = (0..sentences.len()).collect();
indices.shuffle(&mut rng);
```

Sentences are shuffled at the start of each epoch. This prevents the model from seeing sentences in the same order every time, which would cause it to overfit to the sequence of the corpus rather than the content.

7.4 `EpochStats`

```
pub struct EpochStats {
    pub epoch: usize,
    pub avg_loss: f64,
    pub learning_rate: f32,
    pub pairs_processed: u64,
    pub elapsed_secs: f64,
}
```

After each epoch, these statistics are logged (via the `log` crate) and appended to `trainer.history`. This lets you inspect or plot the training progress after the fact. The `save_history()` method serialises this to JSON.

7.5 Progress bars

The `indicatif` crate provides the animated terminal progress bars:

```
let pb = ProgressBar::new(sentences.len() as u64);
pb.set_style(ProgressStyle::with_template(
    "{prefix:.bold} [{elapsed_precise}] [{bar:40.cyan/blue}] {pos}/{len}"
).unwrap());
pb.set_prefix(format!("Epoch {:>3}/{}", epoch + 1, self.config.epochs));
```

Each epoch renders one progress bar showing how many sentences have been processed, elapsed time, and the epoch number.

8 Module: `embeddings.rs`

After training, you interact with the model through the `Embeddings` struct. This is the inference API.

8.1 `get_vector()`

```
pub fn get_vector(&self, word: &str) -> Option<&[f32]> {
    self.vocab.word2idx.get(word)
        .map(|&i| self.model.input_vec(i))
}
```

Looks up the word in the vocabulary to get its index, then returns a slice into the input weight matrix at that row. Returns `None` if the word is not in the vocabulary (out-of-vocabulary / OOV).

8.2 `similarity()`

```
pub fn similarity(&self, word_a: &str, word_b: &str) -> Result<f32> {
    let va = self.get_vector(word_a)...;
    let vb = self.get_vector(word_b)...;
    Ok(cosine_similarity(va, vb))
}
```

Cosine similarity measures the angle between two vectors, ignoring their magnitude:

$$\text{cosine}(\boldsymbol{a}, \boldsymbol{b}) = \frac{\boldsymbol{a} \cdot \boldsymbol{b}}{\|\boldsymbol{a}\| \cdot \|\boldsymbol{b}\|}$$

The result is in $[-1, 1]$. A value of 1 means identical direction (most similar), 0 means orthogonal (unrelated), and −1 means opposite direction.

We use cosine rather than Euclidean distance because embedding magnitudes are not meaningful — a word that appears many times may have a larger-magnitude vector simply because it received more gradient updates, not because it is "more" of anything.

8.3 `most_similar()`

```
pub fn most_similar(&self, query: &str, top_k: usize) -> Vec<(String, f32)> {
    let query_vec = self.get_vector(query)?;

    let mut scores: Vec<(usize, f32)> = (0..self.vocab.len())
        .filter(|&i| self.vocab.idx2word[i] != query)
        .map(|i| (i, cosine_similarity(query_vec, self.model.input_vec(i))))
        .collect();

    scores.sort_unstable_by(|a, b| b.1.partial_cmp(&a.1)...);
    scores.truncate(top_k);
```

```
    // convert indices back to strings
}
```

Computes cosine similarity between the query vector and **every other word's vector** in the vocabulary, then sorts and returns the top k. This is a brute-force O(V × d) scan — perfectly fine for vocabularies up to 100K words. For production at scale, you would replace this with an approximate nearest-neighbour index (HNSW, FAISS).

8.4 `analogy()`

```rust
pub fn analogy(&self, pos_a: &str, neg_a: &str, pos_b: &str, top_k: usize)
    -> Result<Vec<(String, f32)>>
{
    let query: Vec<f32> = (0..dim)
        .map(|i| va[i] - vna[i] + vb[i])
        .collect();
    // find most_similar to query, excluding pos_a, neg_a, pos_b
}
```

The vector arithmetic for analogies. For "king - man + woman":

- `va` = vector for "king"
- `vna` = vector for "man"
- `vb` = vector for "woman"
- `query` = king_vec - man_vec + woman_vec

We then find the word whose vector is closest to `query`, excluding the three input words. If the embeddings are good, this is "queen".

8.5 Save / Load

JSON format — `save()` serialises the entire `Embeddings` struct (model weights + vocabulary) to JSON using `serde_json`. This is human-readable and can be loaded back with `Embeddings::load()`.

Word2Vec text format — `save_text_format()` writes a plain text file:

```
121 50
the  0.123456 -0.234567 0.345678 ...
fox  0.456789 -0.567890 0.678901 ...
```

First line: `vocab_size dim`. Then one word per line followed by its vector. This format is readable by gensim (Python), fastText, and many other tools.

9 Module: `plot.rs`

Generates PNG charts using the `plotters` crate (a pure-Rust charting library).

9.1 `plot_loss_curve()`

Takes the `Vec<EpochStats>` from the trainer and renders a line chart of average loss per epoch. The y-axis is padded 10% above and below the min/max for readability. Each epoch is marked with a filled circle. This gives you an immediate visual check on whether training converged.

9.2 `plot_word_vectors_pca()`

Visualising 50–300 dimensional vectors requires dimensionality reduction. We use **Principal Component Analysis (PCA)** to project down to 2D.

The implementation uses **power iteration** — an iterative algorithm that finds the dominant eigenvector of a matrix without computing a full eigendecomposition (which would be expensive):

```
repeat N times:
    w = X^T (X v)   ← project data onto current estimate, project back
    v = w / ||w||   ← normalise
```

This converges to the first principal component. For the second component, we **deflate** the data by subtracting the projection onto PC1, then run power iteration again.

This is slower than a full SVD but requires no external linear algebra library — the entire implementation is in 40 lines of plain Rust.

The scatter plot renders each word as a coloured dot with its label. Words with similar meanings should cluster visually if training was successful.

10 Binaries

10.1 `bin/train.rs` — CLI Trainer

```
cargo run --release --bin train -- \
  --input corpus.txt  \
  --output embeddings.json \
  --dim 100  --epochs 10  --model skipgram \
  --window 5  --negative 5  --lr 0.025
```

Parses command-line arguments manually (no clap dependency — keeps compile times fast), trains the model, then saves:

- `embeddings.json` — full model
- `embeddings.txt` — text format
- `training_history.json` — per-epoch stats
- `loss_curve.png` — loss chart
- `word_pca.png` — vector projection

Also prints a nearest-neighbour sample for the first 5 vocabulary words so you can immediately sanity-check quality.

10.2 `bin/evaluate.rs` — Interactive REPL

```
cargo run --release --bin evaluate -- --model embeddings.json
> sim king queen
  similarity(king, queen) = 0.6698
> near paris 5
  france    0.8123
  berlin    0.3011
  ...
> analogy king man woman
  king - man + woman ≈
  queen    0.5883
  ...
> quit
```

Loads a saved `embeddings.json` and enters an interactive loop. Useful for exploring embeddings without writing any code.

11 Testing Strategy

11.1 Unit tests (35 total)

Each module has a `#[cfg(test)]` block at the bottom with inline tests. Key examples:

- **`config`**: `default_config_is_valid()`, `zero_dim_is_invalid()` — verify the validation logic.
- **`vocab`**: `vocab_word_counts()`, `vocab_sorted_by_frequency()`, `min_count_filters()`, `noise_table_has_correct_size()` — verify every part of vocabulary construction.
- **`model`**: `loss_decreases_after_repeated_updates()` — trains the same (center, context) pair 200 times and checks that loss at step 200 is lower than step 1. This is the most important unit test — it verifies the gradient math is correct.
- **`embeddings`**: `cosine_identical_vectors()` returns 1.0, `cosine_orthogonal_vectors()` returns 0.0, `similarity_self_is_one()`, `save_load_roundtrip()`.

11.2 Integration tests (10 total)

`tests/integration.rs` runs full training pipelines:

- `loss_decreases_over_epochs()` — trains for 6 epochs, checks that the second-half average loss is lower than the first-half average.
- `similar_words_are_contextually_close()` — trains on sentences about capital cities, checks that paris–berlin similarity exceeds paris–dog similarity.
- `save_and_load_preserves_similarity()` — saves to JSON, loads back, checks that cosine similarities are identical to floating-point precision.
- `text_format_export_creates_valid_file()` — checks the header line of `embeddings.txt` matches `vocab_size` and `embedding_dim`.

11.3 Doc tests (14 total)

Every public function has a `/// # Example` code block that is compiled and run as a test by `cargo test --doc`. This ensures the documentation is always accurate and the example code actually compiles.

12 CI / CD Pipeline

The `.github/workflows/ci.yml` file defines three GitHub Actions jobs:

12.1 Job 1: Check (every push and PR)

```
fmt check → clippy → cargo test --all
```

- `cargo fmt --check` rejects any code that is not formatted according to `rustfmt` rules. This enforces a consistent style without manual review.
- `cargo clippy -D warnings` runs Rust's linter. `-D warnings` means any lint warning is treated as an error, blocking the merge. Clippy catches common mistakes like unnecessary allocations, incorrect use of iterators, and logic errors.
- `cargo test --all` runs unit tests, integration tests, and doc tests.

12.2 Job 2: Docs + Artifacts (main branch only)

After Job 1 passes:

- Builds `rustdoc` HTML documentation with `cargo doc --no-deps`.
- Runs `cargo run --release --example basic_training` to generate fresh `loss_curve.png`, `word_pca.png`, `embeddings.txt`, and `training_history.json`.
- Assembles a static site: rustdoc goes in `site/`, generated artifacts in `site/artifacts/`, and `index.html` (the landing page) at the root.

12.3 Job 3: Deploy (main branch only, after Job 2)

Uses `actions/deploy-pages@v4` to push the assembled site to GitHub Pages. After this runs, the project has a live website with API docs, training plots, and downloadable embeddings.

13 Key Rust Concepts Used

If you are new to Rust, these are the language features that appear most frequently in this codebase.

13.1 Ownership and borrowing

Rust's ownership system ensures memory safety without a garbage collector. Every value has exactly one owner. When you pass a value to a function, you either **move** it (transfer ownership) or **borrow** it (pass a reference). The borrow checker ensures you never have a dangling pointer or data race.

In this codebase: `fn train(&mut self, corpus: &[String])` — `corpus` is an **immutable borrow** (the function can read the corpus but not modify it), while `&mut self` is a **mutable borrow** of the trainer (the function can modify its internal state).

13.2 `Vec<f32>` for matrices

There is no 2D array type in Rust's standard library. We use flat `Vec<f32>` and compute row offsets manually: `row_start = idx * embedding_dim`. This is actually the most cache-friendly representation — a contiguous block of memory.

13.3 `Option<T>` and `Result<T, E>`

Rust has no null pointers and no exceptions. Functions that might fail return `Result<T, E>`. Functions that might not have a value return `Option<T>`. The `?` operator propagates errors upward automatically.

13.4 Traits

Traits are Rust's version of interfaces. Key traits used:

- `Serialize` / `Deserialize` (from `serde`) — automatic JSON serialisation.
- `SeedableRng` (from `rand`) — for seeded random number generators.
- `Display` / `Error` (standard library) — generated by `thiserror`.

13.5 `SmallRng`

A fast, non-cryptographic random number generator. We seed it with `config.seed` to make training reproducible. The same seed, same corpus, same config ⟶ identical trained model.

14 Training Results

Running `cargo run --release --example basic_training` on the built-in 200-sentence corpus (20 unique sentences repeated):

```
Vocabulary: 121 unique tokens (1910 total), model: Skip-gram
Epoch  1/15 | loss: 11.9173 | lr: 0.02334 | pairs:  829
Epoch  2/15 | loss: 10.4022 | lr: 0.02168 | pairs:  820
Epoch  3/15 | loss:  8.3259 | lr: 0.02002 | pairs:  844
Epoch  5/15 | loss:  6.3613 | lr: 0.01670 | pairs:  838
Epoch  8/15 | loss:  4.5066 | lr: 0.01172 | pairs:  826
Epoch 10/15 | loss:  3.8031 | lr: 0.00840 | pairs:  834
Epoch 15/15 | loss:  3.1121 | lr: 0.00010 | pairs:  881
```

Loss drops from 11.9 to 3.1 — a 74% reduction. Nearest neighbours:

```
paris   → european(0.78), france(0.75), major(0.72), city(0.62)
king    → beside(0.76), power(0.71), queen(0.67), royal(0.66)
machine → learning(0.80), branch(0.77), algorithms(0.71)
dog     → fox(0.74), lazy(0.67), jumps(0.76)
```

`machine → learning(0.80)` is a strong result — these two words almost always appear adjacent in the corpus. `king → queen(0.67)` is meaningful despite the tiny corpus size.

Note: These results are from a 20-sentence toy corpus. With a real corpus (e.g. the text8 Wikipedia dataset, 17M tokens), you would see much stronger semantic clustering — the classic king/queen/man/woman analogy typically works well at that scale.

15 What to Do Next

15.1 Train on a real corpus

The toy corpus demonstrates correctness but not quality. Download text8 (100MB cleaned Wikipedia):

```
wget http://mattmahoney.net/dc/text8.zip && unzip text8.zip
python3 -c "
text = open('text8').read().split()
for i in range(0, len(text), 20):
    print(' '.join(text[i:i+20]))
" > corpus.txt

cargo run --release --bin train -- \
  --input corpus.txt --output text8.json \
  --dim 100 --epochs 5 --model skipgram \
  --min-count 5 --negative 5
```

15.2 Evaluate on standard benchmarks

The standard way to evaluate word embeddings is word similarity datasets:

- **WordSim-353** — 353 word pairs with human similarity scores. Compute Spearman correlation between model similarity scores and human scores.
- **SimLex-999** — similar but distinguishes similarity from association.
- **Google Analogy Test Set** — 19,544 analogy questions like "king : man :: queen : ?".

15.3 Add parallelism

Each sentence in the training loop is independent — sentences do not share mutable state during the forward/backward pass. This makes the training loop embarrassingly parallelisable. Adding `use rayon::prelude::*` and changing the sentence loop to `sentences.par_iter()` would use all CPU cores.

The complication is that multiple threads would write to the same model weights simultaneously. The original word2vec C code does this without locks (Hogwild-style asynchronous SGD) and it works well in practice because collisions are rare and the noise is tolerable.

15.4 Use embeddings in Python

```
from gensim.models import KeyedVectors

wv = KeyedVectors.load_word2vec_format("embeddings.txt", binary=False)
print(wv.most_similar("king", topn=5))
print(wv.similarity("paris", "berlin"))
```

The `embeddings.txt` file produced by `save_text_format()` is directly loadable by gensim.

16 Summary

`error.rs`	Unified error type with `thiserror`. All functions return `Result<T, Word2VecError>`.
`config.rs`	All hyperparameters in one struct with sensible defaults and a `validate()` guard.
`vocab.rs`	Frequency counting, `min_count` filtering, Mikolov subsampling, 1M-entry unigram noise table for O(1) negative sampling.
`model.rs`	Two flat `Vec<f32>` weight matrices. Xavier init. Skip-gram and CBOW SGD update with in-place gradient accumulation. Dynamic window.
`trainer.rs`	Full training loop with linear LR decay, sentence shuffling, subsampling, progress bars, and epoch history recording.
`embeddings.rs`	Post-training API: cosine similarity, brute-force nearest neighbours, vector arithmetic analogy, JSON and text-format save/load.
`plot.rs`	Loss curve PNG and 2D PCA scatter plot using `plotters`. PCA via power iteration — no external linear algebra dependency.
`bin/train`	Full-featured CLI. Saves model, text vectors, history JSON, and both plots after training.
`bin/evaluate`	Interactive REPL for `sim`, `near`, and `analogy` queries against a saved model.

The full codebase is approximately 1,500 lines of Rust across 8 modules, with 45 tests (35 unit + 10 integration), all passing. Every public function has doc-test examples. The CI pipeline enforces formatting, linting, and tests on every commit, and deploys live documentation and training artifacts to GitHub Pages on every merge to main.